

Boiler & Chimney IoT — Day 1 & Day 2 Walkthrough

A senior dev explaining it to you, line by line

Who you are: You know Python (variables, functions, classes, `pip install`). You have never touched MQTT, Docker, or IoT before. **What you'll have after Day 2:** A simulated factory running on your laptop — a fake boiler and fake chimney sending sensor readings every half-second through a real industrial message broker, including occasional "faults" (broken equipment events).

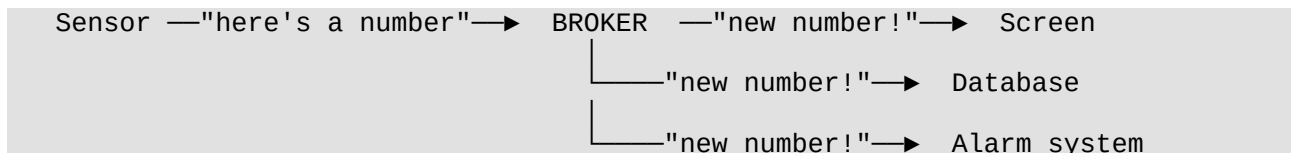
□ Before You Touch Any Code — The Mental Model

Forget code for two minutes. Picture a real factory:

- A **boiler** has sensors bolted onto it (thermometers, pressure gauges, water-level floats).
- Each sensor produces a number every fraction of a second.
- Somewhere far away, an operator watches a screen showing those numbers live.
- If pressure suddenly spikes, an alarm fires.

The question every IoT system answers: **how do numbers get from a sensor on a machine to a screen in an office, reliably, even if there are 1,000 sensors and 50 screens?**

The answer in this industry is a pattern called **publish/subscribe (pub/sub)** with a **broker** in the middle:



The sensor doesn't know who's listening. The screen doesn't know which sensor sent the number. They both only talk to the broker. That's it. That's the whole idea.

MQTT is just the specific "language" (protocol) they use to talk to the broker. **EMQX** is the specific broker software we'll run. Once you understand pub/sub, everything else is just plumbing.

📅 DAY 1 — Get the Plumbing Working

What "done" looks like at the end of today

1. Docker is installed and you can run things in it.
2. EMQX (the broker) is running on your laptop and you can open its admin page in a browser.
3. You wrote two Python scripts — one that sends test messages, one that receives them — and you watched messages flow in real time.

That's it. No boiler yet. Today is just: **prove the pipes work**.

Step 1.1 — Install the tools

You need three things. Install them in this order.

(a) Docker Desktop

Download from <https://www.docker.com/products/docker-desktop>, install, and **launch the Docker Desktop app** (the whale icon must be in your system tray). Docker won't work until that app is running.

Why Docker? EMQX is complicated software. Installing it directly on Windows is painful. Docker lets us run it inside a sealed "container" — a mini-Linux-machine inside your Windows machine — with one command. Same goes for InfluxDB, Grafana, Kafka later. Docker is how modern devs avoid "it works on my machine" hell.

Verify:

```
docker --version
docker-compose --version
```

If both print versions, you're good. If `docker - compose` is "command not found," try `docker compose` (newer Docker Desktop bundles it as a subcommand — both forms work).

(b) Python 3.10 or newer

You already have this. Check:

```
python --version
```

(c) The Python MQTT library

We need a Python library that knows how to speak MQTT. The standard one is `paho-mqtt`.

```
# inside your project folder
python -m venv .venv
.venv\Scripts\activate          # Windows
# source .venv/bin/activate     # Mac/Linux
pip install paho-mqtt
```

Why a virtual environment (venv)? Each project gets its own isolated copy of Python packages. If you `pip install` globally, your projects fight over package versions. `.venv` keeps this project's dependencies in a folder you can delete and rebuild any time. It's a habit — always use one.

Your repo already has `.venv` and `pyproject.toml`, so you may already be set up. Just run `.venv\Scripts\activate` and `pip install paho-mqtt`.

Step 1.2 — Run the EMQX broker

We could run EMQX with one long `docker run . . .` command, but that's annoying to remember and edit. Instead we use **docker-compose** — a YAML file that describes everything we want running, so we can start it all with one command.

Create a file called `docker-compose.yml` in your project root:

```
version: '3.8'

services:
  emqx:
    image: emqx/emqx:latest           # which prebuilt image to download
    container_name: emqx_broker       # the running container's name
    ports:
      - "1883:1883"                   # MQTT (our Python scripts use this)
      - "8083:8083"                   # MQTT over WebSocket (browsers)
      - "18083:18083"                 # EMQX's own admin web UI
    environment:
      - EMQX_ALLOW_ANONYMOUS=true     # no username/password in dev
    volumes:
      - emqx_data:/opt/emqx/data      # persist data across restarts
    restart: unless-stopped

volumes:
  emqx_data:
```

What's 1883:1883? Left side = port on YOUR laptop. Right side = port inside the container. So when your Python script connects to `localhost:1883`, Docker forwards it into the container's port 1883 where EMQX is listening. Three ports, three different things EMQX speaks.

EMQX_ALLOW_ANONYMOUS=true is a dev shortcut — no passwords. In production you'd never do this.

Start it:

```
docker-compose up -d emqx
```

`-d` means "detached" — runs in the background.

Now open your browser: **`http://localhost:18083`** Login: `admin` / `public` (it'll force you to change the password on first login — pick anything).

You should see the EMQX dashboard. This is the control room for your broker. Click around. You'll come back to the **Topics** tab later to watch messages live.

Step 1.3 — Your first publisher

Now the fun part. We'll write a tiny script that sends a message to the broker every second.

Create `day1_publisher.py`:

```
"""Day 1 Publisher — sends a test message every second."""

import paho.mqtt.client as mqtt
import json
```

```

import time

BROKER_HOST = "localhost"
BROKER_PORT = 1883
TOPIC = "test/hello"
CLIENT_ID = "publisher_001"

def on_connect(client, userdata, flags, rc):
    if rc == 0:
        print("Connected to EMQX broker")
    else:
        print(f"Connection failed with code: {rc}")

client = mqtt.Client(client_id=CLIENT_ID)
client.on_connect = on_connect
client.connect(BROKER_HOST, BROKER_PORT, keepalive=60)
client.loop_start()
time.sleep(1)

message_count = 0
while True:
    message_count += 1
    payload = {
        "message_id": message_count,
        "text": "Hello from boiler system!",
        "timestamp": time.time(),
    }
    client.publish(TOPIC, json.dumps(payload), qos=1)
    print(f"Published #{message_count}")
    time.sleep(1)

```

Walking through this line by line

- **`mqtt.Client(client_id=...)`** — every client connected to the broker needs a unique name. The broker uses it to track who's who. Two clients with the same ID will kick each other off.
- **`on_connect`** — a *callback*. You don't call it; `paho` calls it for you when the connection finishes. `rc=0` means success; any other number is an error code.
- **`client.connect(...)`** — opens the TCP connection.
- **`client.loop_start()`** — this is critical and weird. MQTT runs on a background thread for network I/O. Without this line, no messages will actually leave your script. `loop_start()` spins up that background thread. (There's also `loop_forever()` for subscriber scripts — we'll see that next.)
- **`time.sleep(1)` after connect** — give the connection a moment to establish before publishing. Without it, your first message might be sent before you're actually connected.
- **`payload = {...}` then `json.dumps(...)`** — MQTT itself doesn't care what your payload looks like; it just ships bytes. The industry convention is to send JSON. Every consumer downstream (database, dashboard, LLM) speaks JSON.
- **`client.publish(TOPIC, payload, qos=1)`** — finally, the actual send.
 - `TOPIC = "test/hello"`. This is the *address* of the message. Anyone subscribed to `test/hello` will receive it.

- `qos=1` = "Quality of Service 1" = "at-least-once delivery." Means the broker keeps retrying until it gets an acknowledgement. `qos=0` is fire-and-forget (faster, but messages can be lost). `qos=2` is exactly-once (slowest, used for critical events like fault alerts).
-

Step 1.4 — Your first subscriber

Open a **second terminal** (keep the publisher running). Create `day1_subscriber.py`:

```
"""Day 1 Subscriber – listens on a topic and prints what arrives."""

import paho.mqtt.client as mqtt
import json

BROKER_HOST = "localhost"
BROKER_PORT = 1883
TOPIC = "test/hello"
CLIENT_ID = "subscriber_001"

def on_connect(client, userdata, flags, rc):
    if rc == 0:
        print("Subscriber connected")
        client.subscribe(TOPIC, qos=1)
        print(f"Subscribed to {TOPIC}")

def on_message(client, userdata, message):
    payload = json.loads(message.payload.decode("utf-8"))
    print(f"Received on [{message.topic}]: {payload}")

client = mqtt.Client(client_id=CLIENT_ID)
client.on_connect = on_connect
client.on_message = on_message
client.connect(BROKER_HOST, BROKER_PORT, keepalive=60)
client.loop_forever()
```

What's different from the publisher

- `client.subscribe(TOPIC, qos=1)` is called *inside* `on_connect`. Why? Because you can't subscribe until you're connected. Putting it in the callback guarantees ordering — and crucially, if you ever lose the connection and reconnect, the callback runs again and re-subscribes automatically. Free reliability.
- `on_message` is a new callback. Whenever a message arrives on a subscribed topic, paho calls this for you.
- `message.payload` is raw bytes. `.decode("utf-8")` gives you a string. `json.loads(...)` parses the JSON string into a Python dict.
- `client.loop_forever()` instead of `loop_start()` — this is the *blocking* version. The script sits here forever, processing messages. Perfect for a subscriber whose whole job is to listen.

Run them

Terminal 1: `python day1_subscriber.py` Terminal 2: `python day1_publisher.py`

You should see messages flowing. Now go open the EMQX dashboard → **Monitoring** → **Topics** → you'll see `test/hello` with traffic on it. **This view is your best debugging friend.** When something breaks later, "is the message actually reaching the broker?" is the first question, and this page answers it.

☐ Day 1 Checklist

- ☐ `docker-compose up -d emqx` runs without errors
- ☐ `http://localhost:18083` loads the EMQX dashboard
- ☐ Publisher prints "Published #N" every second
- ☐ Subscriber prints "Received on [test/hello]: ..."
- ☐ You can explain in one sentence what a topic, publisher, subscriber, and broker are

If any box is unchecked, stop and fix it. Day 2 builds directly on top of this.

DAY 2 — Simulate a Real Boiler and Chimney

What "done" looks like at the end of today

1. A `boiler_simulator.py` running that publishes **8 sensors** worth of realistic data twice per second.
 2. A `chimney_simulator.py` doing the same for **7 chimney sensors**.
 3. Occasional "fault" events appearing on a special `system/faults` topic (this is what the AI will learn to detect later).
 4. You can watch all of it live on the EMQX dashboard.
-

☐ The Concept — What Makes Simulated Data "Realistic"?

A naive simulator does this:

```
temperature = random.uniform(60, 100)
```

Every reading is a totally random jump. That's not how real sensors behave. Real readings have three properties we need to mimic:

1. **Slow drift** — temperature might trend from 85 → 87 → 89 over a minute, not jump randomly.
2. **Small noise** — each reading wobbles slightly around the "true" value (sensor jitter).

3. **Occasional faults** — sometimes equipment breaks and a value goes wildly outside normal range. The AI later has to learn what "wildly outside" looks like.

Our simulator uses `math.sin()` for slow drift (a smooth oscillation), `random.gauss()` for noise (small bell-curve wobble), and a **5% random chance per tick** to inject a fault. This produces data that *looks* like real industrial telemetry.

Step 2.1 — Project structure

Create folders:

```
boiler-iot-system/
├── docker-compose.yml
├── day1_publisher.py
├── day1_subscriber.py
├── simulators/
│   ├── boiler_simulator.py
│   └── chimney_simulator.py
```

Step 2.2 — The Boiler Simulator

Create `simulators/boiler_simulator.py`. The full code is in `CLAUDE.md` lines 389–591 — copy it from there. Here I'll explain **what each section is doing and why**, so when you read the code it makes sense.

Section 1: Topic map

```
TOPICS = {
    "temperature": "boiler/temperature",
    "pressure":    "boiler/pressure",
    ...
}
```

One sensor → one topic. Why not put everything on a single `boiler/all` topic? Because **subscribers should be able to filter cheaply**. A dashboard that only cares about pressure subscribes to `boiler/pressure` and never wastes bandwidth on temperature data. This is also why MQTT topics use `/` like folders — a subscriber can say `boiler/#` to mean "everything under boiler."

Section 2: Normal operating ranges

```
NORMAL = {
    "temperature": {"min": 60, "max": 100, "unit": "C", "mean": 85},
    "pressure":    {"min": 10, "max": 14, "unit": "bar", "mean": 12},
    ...
}
```

This is the *physical reality* of how a boiler operates. A real boiler's pressure is between 10 and 14 bar — anything outside is a problem. We hardcode these so the simulator knows what "normal" looks like and what "fault" looks like.

Section 3: Fault definitions

```
FAULT_TYPES = {
    "HIGH_PRESSURE": {"sensor": "pressure", "multiplier": 1.3, "severity":
"CRITICAL"},
    "LOW_WATER_LEVEL": {"sensor": "water_level", "multiplier": 0.5, "severity":
"CRITICAL"},
    ...
}
```

When a fault fires, we take the sensor's normal mean and multiply it. `multiplier=1.3` on pressure means pressure jumps to $12 \times 1.3 = 15.6$ bar — well above the 14 bar safe maximum. That's a critical fault.

Section 4: The `BoilerSimulator` class

It tracks three things:

- **`self.state`** — current value of every sensor right now.
- **`self.active_fault`** — name of the fault currently happening (or `None`).
- **`self.fault_duration`** — how many more ticks this fault lasts before resolving.

The class has three key methods:

`_drift_value(sensor)` — produces the next reading:

```
drift = math.sin(self.tick * 0.05) * (cfg["max"] - cfg["min"]) * 0.05
base = cfg["mean"] + drift
return self._add_noise(base)
```

`math.sin(self.tick * 0.05)` returns a value oscillating between -1 and 1, slowly.

Multiply by a small fraction of the sensor's range → slow gentle drift. Then `_add_noise` adds tiny Gaussian wobble. That's "realistic-looking" sensor data in five lines.

`_inject_fault()` — 5% chance each tick to roll a new fault, picks one at random, sets it active for 10–30 ticks. Only one fault at a time.

`update()` — called once per tick (every 500 ms). Drifts every sensor, then if a fault is active, overwrites the affected sensor with the fault value. Decrements the fault timer; when it hits zero, the fault clears.

Section 5: The main loop

```
while True:
    new_fault = simulator.update()
    timestamp = datetime.utcnow().isoformat() + "Z"

    for sensor_name, topic in TOPICS.items():
        ...
        payload = {
            "device_id": "BOILER_001",
            "sensor": sensor_name,
            "value": value,
            "unit": ...,
            "timestamp": timestamp,
            "status": simulator.get_status(),
```



```
}
client.publish(topic, json.dumps(payload), qos=1)

if new_fault:
    client.publish(TOPICS["fault"], json.dumps(fault_payload), qos=2)

time.sleep(0.5)
```

Five details that matter

1. **`datetime.utcnow().isoformat() + "Z"`** — every payload carries a timestamp in ISO-8601 UTC (e.g. `2026-06-09T10:23:45.123Z`). InfluxDB will use this exact timestamp tomorrow. Always include it.
 2. **`"device_id": "BOILER_001"`** — when you scale to multiple boilers, you need to know which one sent the data. Bake it in from day one.
 3. **`qos=1` for sensor data, `qos=2` for faults.** Losing one temperature reading out of 100 doesn't matter. Losing a *fault alert* could be catastrophic. Match QoS to the cost of losing the message.
 4. **`time.sleep(0.5)`** — 2 messages per second, per sensor. With 8 sensors that's 16 messages/sec, plus a status summary, plus occasional faults. Tomorrow's InfluxDB consumer needs to keep up with this.
 5. **One topic per sensor + one summary topic + one fault topic.** This layout repeats for the chimney. It's a design pattern — copy it for any new device.
-

Step 2.3 — The Chimney Simulator

Copy `simulators/chimney_simulator.py` from `CLAUDE.md` lines 599–721. The structure is **identical** to the boiler — same class shape, same loop, just different sensor names and fault types. That repetition is intentional: the pattern works, so don't reinvent it.

Chimney-specific sensors:

- `flue_temp` — exhaust gas temperature
- `co2`, `o2`, `co` — combustion chemistry
- `draft` — pressure differential pulling smoke up the chimney (always slightly negative)
- `stack_velocity` — how fast gas moves up the chimney

Chimney-specific faults:

- `BLOCKED_FLUE` — draft drops to 30% of normal (smoke can't escape — dangerous)
 - `HIGH_CO` — carbon monoxide spikes 5× (incomplete combustion — toxic)
-

Step 2.4 — Run both simulators together

You need **two terminals** (three, if you want to keep a subscriber up to spy on traffic).

Terminal 1:

```
python simulators/boiler_simulator.py
```

Terminal 2:

```
python simulators/chimney_simulator.py
```

Open EMQX dashboard → **Monitoring** → **Topics**. You should see ~15 topics, each with a live message rate.

Want to watch a specific topic live? Modify `day1_subscriber.py` to subscribe to `boiler/#` instead of `test/hello`. The `#` wildcard means "everything under boiler/", so you'll see all 8 boiler sensors stream in one terminal.

☐ Day 2 Checklist

- ☐ `boiler_simulator.py` runs and prints occasional `⚠ FAULT` lines
 - ☐ `chimney_simulator.py` runs in parallel
 - ☐ EMQX dashboard shows ~15 active topics with steady traffic
 - ☐ A subscriber on `boiler/#` receives data from all 8 boiler sensors
 - ☐ A subscriber on `system/faults` occasionally sees a fault event
 - ☐ You can explain *why* `math.sin` is used for drift and *why* faults use higher QoS
-

What You Actually Learned

After two days you understand:

1. **Pub/sub architecture** — the foundation of every modern IoT, messaging, and event-driven system. (The same pattern shows up in Kafka, RabbitMQ, AWS SNS, Redis Pub/Sub — they're all variations on this idea.)
2. **Docker basics** — running services without installing them on your OS.
3. **MQTT specifics** — topics, QoS levels, wildcards, callbacks, the importance of `loop_start` vs `loop_forever`.
4. **Realistic simulation** — drift + noise + random faults. This same pattern is used for testing ML models, load-testing APIs, training reinforcement-learning agents — anywhere you need fake-but-plausible data.

Tomorrow (Day 3) we plug InfluxDB and Grafana into this pipeline and turn those flowing numbers into a live dashboard. Make sure both simulators have been running for at least an hour

before starting Day 3 — Day 3 needs accumulated data to look interesting.